

DIMENSIONAL DATA MODELLING FOR RETAIL BANKING ANALYTICS

TECHNICAL REPORT (HNG STAGE 3 TASK B)

1.0 DATA PROFILING AND DIMENSION IDENTIFICATION

The source dataset from Palladium Bank contains 15 columns representing 18 months of raw transaction data. After profiling each column,

1.1 Classify columns as Dimension, Fact or Degenerate Dimension.

What is Dimension, Fact and Degenerate Dimension

- Dimension** most times represent the who, what, where, when and why of a business. They are usually descriptive and they also have context.
- Fact** represents the numerical measures that are link to the dimension tables. It is at the center of everything. It represents actions or business events.
- Degenerate Dimension** They represent dimensions that has little to no content except they primary key and they are usually stored within the fact table rather than having their own table.

Dimension	Fact	Degenerate Dimension
Customer : Customer_ID, Customer_Name, Tier	Amount	Txn_ID
Branch: Branch_ID, Branch_Name, State.	Balance_After	
Product: Product_ID, Product_Name, Product_Type.		
Category: Txn_Type, Channel, Txn_Date.		

1.2 Define each Dimension, its attributes, and the surrogate key needed.

Dimension	Definition	Attributes	Surrogate Key Need
Dim_customer	Bank clients who perform the transaction	Customer_Name, Tier, Customer_ID, Customer_Key	This is needed for SCDs' history tracking because a customer might move or change tiers over time. This helps to keep things accurate.
Dim_Product	Contains information about specific bank products, such as savings and investments, being used.	Product_Key, Product_ID, Product_Name, Product_Type	It is used to improve query performance and they can be update, surrogate key maintains transparency.
Dim_Date	Contains the specific transaction date broken down into a smaller time frame.	Date_Key, full_date, Day, Month, Month_name, Quarter, Year, is_weekend, is_public_holiday.	Enables fast filtering and aggregation for time-based data.
Dim_channel	This is used to identify the platform or	Channel_Key, Channel_Name	It allows for easy channel report and provides an avenue for

	medium through which the transaction was conducted.		tracking volume and trend of different channels.
Dim_Branch	Represent the place or location which in which the transaction was carried out.	Branch_Key, Branch_ID, Branch_Name, State.	Branch names could be renamed or rebranded anytime; this allows for a stable identifier in case of anything.

1.3 Decide and justify the fact table grain

When it comes to Data Modelling, deciding on a grain to use all comes down to how much details we keep versus speed or storage we want. There are different types of fact grains but I will be using the **Per Transaction fact grain**.

What is the Per Transaction Fact Grain: This is the most common fact grain used in banking today. According to the Kimball Group It is the most detailed level of data in a dimensional model, representing one row for every individual business event, such as single sale, bank transaction or click.

Reason as why I am using Per Transaction Fact Grain (Justification)

I picked the Per Transactional Grain because some of my business objectives involves the analysis of churn signals and behavior analysis. Churn signals involve tracking recency and frequency which per transactional grain can do. I need to know exactly when last a customer used the bank every single transaction needs to be store and documented, since transactional grain has the most detailed data model. It is the best fact grain for me in this scenario.

1.4 Identify degenerate dimensions and how they'll be handled

Degenerate Dimension	How they'll be handled
Txn_ID	It will be stored directly in the fact transaction model table with no dimension table created. Txn_ID is a unique transaction reference with no additional attributes.

2.0 STAR SCHEMA DESIGN

A star schema was designed consisting of one central fact table — Fact_Transactions — surrounded by five-dimension tables: Dim_Date, Dim_Customer, Dim_Branch, Dim_Product and Dim_Channel. Each dimension table was assigned a surrogate primary key using AUTO_INCREMENT to ensure performance and historical accuracy. The complete SQL schema including all primary keys, foreign keys, surrogate keys and comments is provided in the attached SQL file.

3.0 SLOWLY CHANGING DIMENSIONS (SCD)

3.1 Identify which dimension might change over time

Tiers

State

Product Name

Product type

Branch name

Customer Name

3.2 Chose and Justify SCD strategy (e.g Type 1 or Type 2) for at least two dimension

SCD strategy can either be type 1 or type 2. SCD also referred to as Slowly Changing Dimension shows how Dimension data changes over time and here are the common types.

SCD Type 1: the overwrite

This method simply overwrites the old data with new data. it does not keep history of the old data the moment the dimension changes. Good for when you want your table to be small but history will be deleted.

SCD type 2 also known as 'the history' this method create a new row for every dimension data change instead of wiping the history. Good for keeping historical data accurately, but the data becomes large due to adding information to information.

3.2.1: State –

Strategy- Type2

Justification

A branch's state classification could change due to administrative redistricting or bank regional reorganization. Type 2 is applied to preserve historical regional performance data for accurate geographical analysis

3.2.2. Tier

Strategy - Type 2

Justification

Customers increase their tiers often over time due to maybe promotions from the bank. Keeping historical data for tiers is also important because it is crucial for tracking the loyalty of customers per tiers.

3.2.3. Branch_Name

Strategy - Type 1

Justification

Usually happens as a rebrand, and it doesn't really have any historical influence so it can be overwritten, and the rebranded branch name can be used.

3.2.4 Product_Name

Strategy – Type 1

Justification

Product names change rarely and only due to rebranding. Historical product name changes have no significant analytical impact; analysts care about the current product name. Overwriting is sufficient.

4.0. ETL / ELT Strategy

4.1 Initial Load

The initial load is a one-time migration of 18 months of historical data from source files into the star schema.

4.1.1 Steps to load 18 months of data: To load 18 months of historical data, I will start the process by extracting raw CSV files into a temporary storage space, like Snowflake stages, to avoid disrupting the main system. Once it is there, the data will undergo a cleaning phase to standardise the dates, remove currency symbols, and fix text inconsistencies.

Next, I will focus on unique business entities, such as customers and products, which are filtered out and loaded into their respective Dimension tables to establish the necessary reference points. Finally, the transactions are matched to these dimensions and loaded into the *Fact_transaction* table in monthly batches. These importing of data ensures smooth processing without overloading the system.

4.1.2. Order of Table Population:

To maintain data integrity, we load descriptive tables first and the event table last:

1. **Dim_Date** (Calendar dates)
2. **Dim_Customer, Dim_Branch, Dim_Product, Dim_Channel**(Core business entities)
3. **Fact_Transactions** (Loaded last to ensure all foreign keys exist)

4.1.3. Required Transformations:

1. **Standardise Dates:** Convert various formats (e.g., 15/01/24) to the standard YYYY-MM-DD.
2. **Clean Amounts:** Remove currency symbols (₹) and commas to convert values into decimals.
3. **Surrogate Keys:** Automatically generate unique ID numbers for each record.
4. **Text Formatting:** Standardise labels (e.g., change "mobile" or "MOBILE" to "Mobile").

4.2 Incremental Loads

After the initial load, new data is added daily using an incremental strategy.

- 4.2.1. **Detect New Records:** The system compares incoming data against existing records and only inserts **Txn_IDs** that have not been seen before.
- 4.2.2. **Handle SCD Updates:** For changes like a customer moving to a new "Tier," the system "expires" the old record with a timestamp and inserts a new "current" record.

4.2.3. Prevent Duplicate Facts: A unique constraint is placed on the **Txn_ID** column, which physically blocks the same transaction from being saved twice.

4.3 Data Quality Checks

4.3.1. Null Amount Check

- **What:** Ensures no transaction is missing its price.
- **How to detect:** Scans for "blank" or empty values in the Amount column.
- **Action:** Reject the record and log it in an error table.

4.3.2. Date Range Check

- **What:** Ensures dates fall within the 18-month window and are not in the future.
- **How to detect:** Flags dates older than 2023 or later than today.
- **Action:** Flag as suspicious and alert the data team.

4.3.3. Duplicate ID Check

- **What:** Ensures every transaction has a unique identity.
- **How to detect:** Checks if a **Txn_ID** appears more than once in the source.
- **Action:** Load only the first occurrence and discard the rest.

4.3.4. Customer Reference Check

- **What:** Ensures the transaction belongs to a known customer.
- **How to detect:** Matches the incoming **Customer_ID** against the Customer Table.
- **Action:** Pause the record until the missing customer is added to the system.

5.0 HIERARCHIES, PERFORMANCE AND SCALABILITY

5.1. Hierarchies – Defining drill down structures

Hierarchies	Drill down Structure
Date/Time	Year → Quarter → Month → Day
Location	State → Branch
Product	Product_Type → Product_Name
Customer	Tier → Customer_Name

5.2 Performance and Scalability

5.2.1 Partitioning Strategy: Monthly Range Partitioning

Recommendation: Instead of putting all 18 months of transactions into one giant pile, I have split them into monthly folders using horizontal Range Partitioning on the **Txn_Date** column of the **Fact_Transactions** table.

Justification: Bank transaction data is usually time-series like looking for a specific receipt from March. Instead of searching through a huge box of 18 months of paperwork, the computer goes straight to the "March" folder and ignores everything else. This makes

finding data much faster and ensures that query performance remains stable even as the dataset grows over the coming years.

5.2.2 Indexing Approach: Composite Indexes

Recommendation: I have added digital "bookmarks" to the most important parts of the records, like Dates, Customers, Branches and Channel. It is also a way to create a composite index for common queries with foreign Key Indexes (Date_Key, Customer_Key, Branch_Key, Channel_Key)

Justification: It works like the index at the back of a book. Instead of reading every single page to find information about a specific customer or branch, the system can jump exactly to the right page. I also made a special "double bookmark" for Date and Branch so that regional reports load instantly.

5.2.3. Aggregation Table: Monthly Branch Revenue Summary

Recommendation: I created a pre-calculated table named Agg_Monthly_Branch_Revenue. This aggregate table works like a "cheat sheet."

Justification: Normally, to see how a branch is doing, the computer has to add up millions of individual sales every time you click "Refresh." With this cheat sheet, the computer calculates the totals (like total deposits and withdrawals) once a month and saves the answer.